

Object Detection

Note: If you are using the factory-installed image, there is no need to set up the environment from scratch; the environment is already configured. Simply follow the tutorial to execute the corresponding commands to utilize the features.

1. Object Prediction: Images

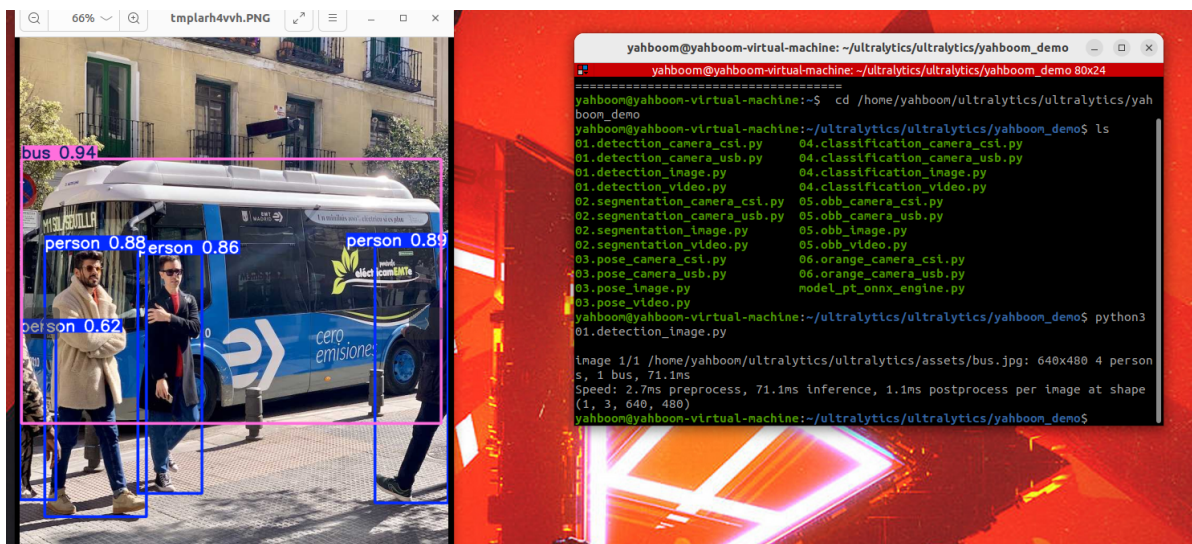
Use `yo1o11n.pt` to perform predictions on the sample images included within the `ultralytics` project.

Navigate to the code directory:

```
cd /home/yahboom/ultralytics/ultralytics/yahboom_demo
```

Run the code:

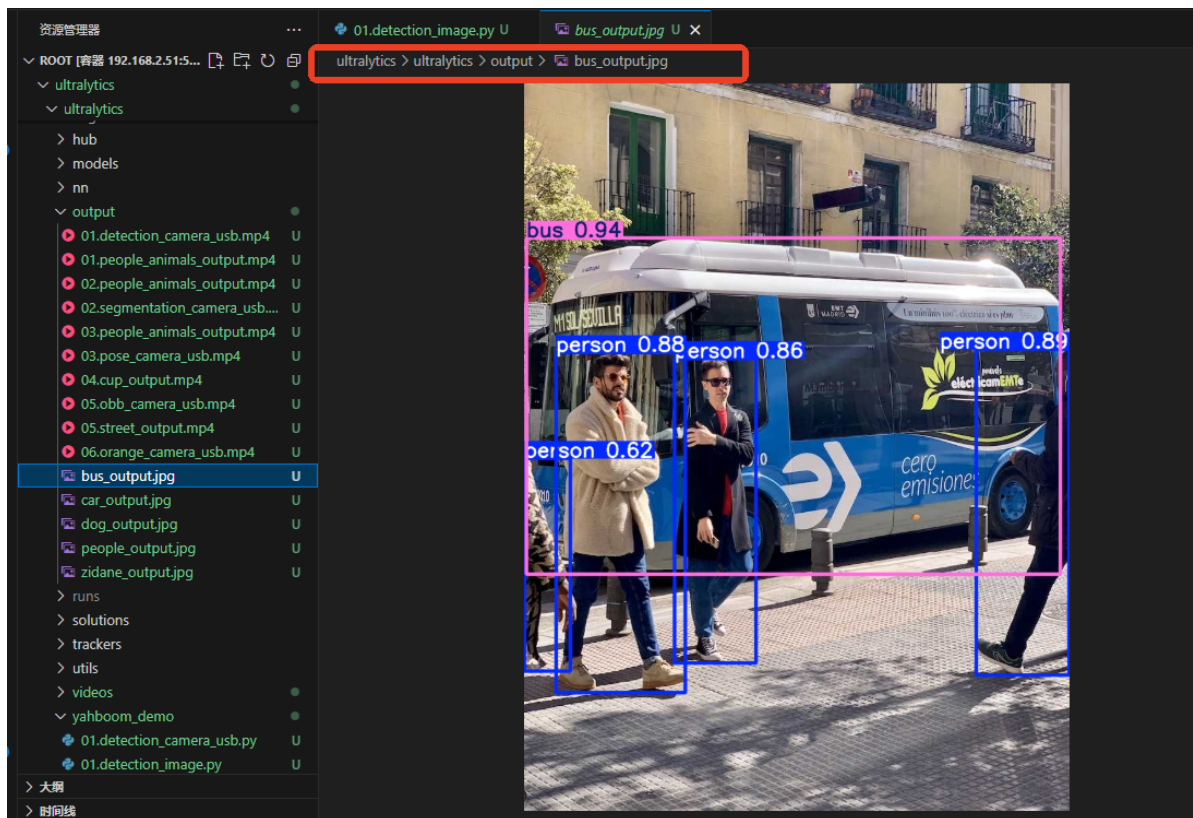
```
python3 01.detection_image.py
```



Preview of Results

Location of the output image generated by YOLO detection:

```
/home/yahboom/ultralytics/ultralytics/output/bus_output.jpg
```



Sample Code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("~/ultralytics/ultralytics/yolo11n.pt")

# Run batched inference on a list of images
results = model("~/ultralytics/ultralytics/assets/bus.jpg") # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    # keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="~/ultralytics/ultralytics/output/bus_output.jpg") #
```

2. Object Prediction: Video

Use `yolo11n.pt` to perform predictions on a video located within the `ultralytics` project directory (specifically, a video *other than* the sample videos bundled with Ultralytics).

Navigate to the code directory:

```
cd /home/yahboom/ultralytics/ultralytics/yahboom_demo
```

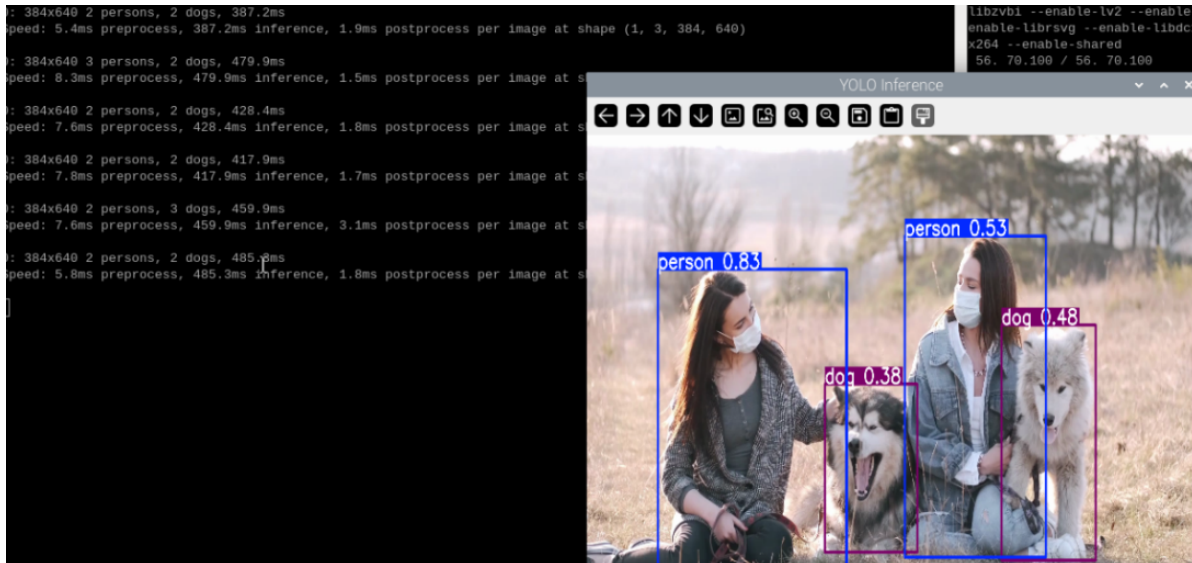
Run the code:

```
python3 01.detection_video.py
```

Preview of Results

Location of the output video generated by YOLO detection:

```
~/home/yahboom/ultralytics/ultralytics/output/
```



Sample Code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("~/ultralytics/ultralytics/yolo11n.pt")

# Open the video file
video_path = "~/ultralytics/ultralytics/videos/people_animals.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "~/ultralytics/ultralytics/output/01.people_animals_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()
```

```

# Write the annotated frame to the output video file
out.write(annotated_frame)

# Display the annotated frame
cv2.imshow("YOLO Inference", annotated_frame)

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the end of the video is reached
    break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

3. Object Prediction: Real-time Detection

3.1. Launching the Camera

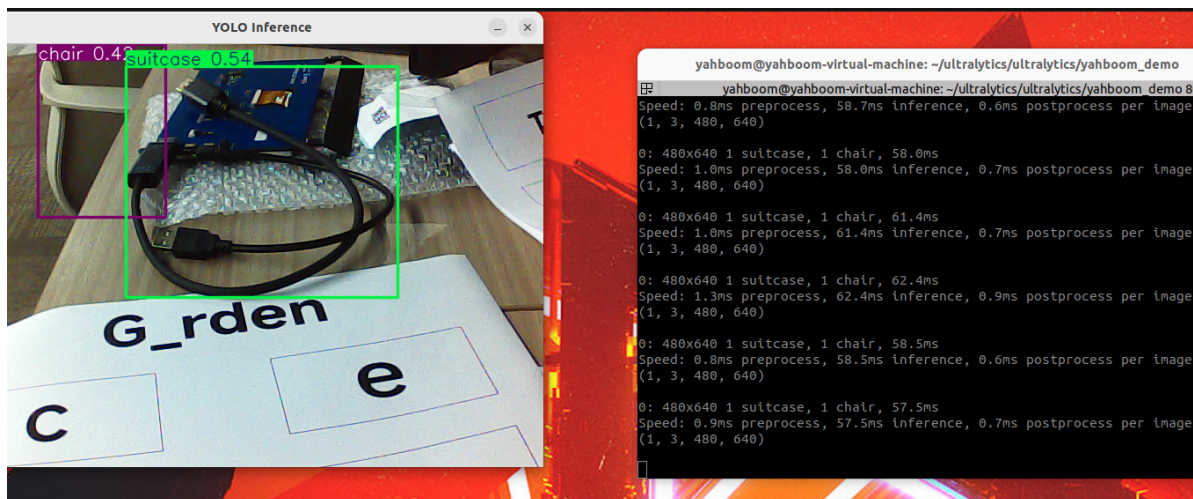
Access the Docker container terminal and navigate to the code directory:

```
cd /home/yahboom/ultralytics/ultralytics/yahboom_demo
```

Run the code: Click on the preview window, then press the 'q' key to terminate the program!

```
python3 01.detection_camera_usb.py
```

Preview of Results



Example Code Path:

/home/yahboom/ultralytics/ultralytics/yahboom_demo/01.detection_camera_usb.py

